

AI Academy, National AI Center
AI-ENG-110 – Software Engineering

Async Research Assistant

VerteX

Yaqub Xəlilli Səbuhi Xamiyev Aqil Əsgərov

github.com/yagubbx/Research-Assistant

The research contract

User outcome

A question becomes a concise answer with inspectable numeric references and visible missing-source notes.

Input

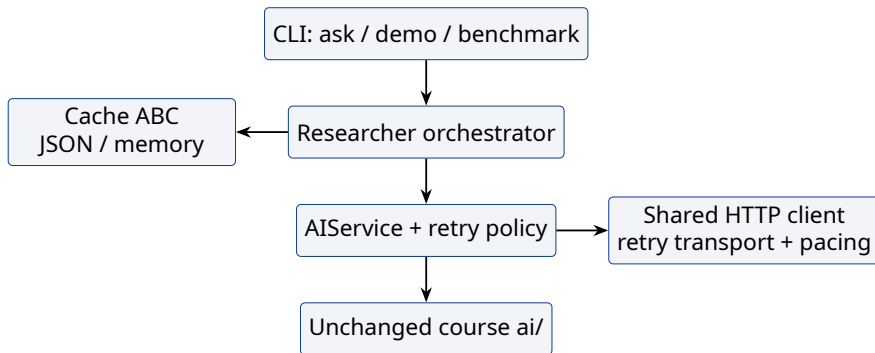
- ▶ Question: 1–2,000 characters
- ▶ Selected sources: wiki, arxiv, web
- ▶ Optional cache bypass

Output

- ▶ Cited answer and source URLs
- ▶ Per-source status and timing
- ▶ Plain text or structured JSON

```
python -m researcher ask "your question"
```

Architecture



Pydantic models cross boundaries. Only the service and its worker call the supplied AI package. No changes to `ai/`.

Design decisions

1. **Async I/O for retrieval**

Independent sources overlap; a semaphore caps active operations.

2. **Filesystem cache behind an interface**

Persistence without a database server; memory backend supports tests.

3. **A process for synchronous synthesis**

A deadline can terminate the SDK call; startup adds latency.

Retrieval benchmark

Five questions, three repetitions, cache bypassed

	Sequential	Parallel
Median seconds	2.412	0.802
Speedup	1.00×	3.01×
Failed / empty	0	0

Synthetic HTTP delays: 150 ms per source; semaphore = 3.

Scope: retrieval only; no live-provider latency claim.

Bottleneck: the slowest source after overlap; synthesis may dominate the full live run.

Failure handling

Injected incident: arXiv raises a provider error

- ▶ Wikipedia and web evidence still reaches synthesis.
- ▶ References include an explicit arXiv-unavailable note.
- ▶ Transient HTTP failures retry; 401 does not.
- ▶ All sources unavailable: clear error, no invented answer.

Defaults: 3 attempts; 0.5 / 1.0 s backoff; 20 s source deadline; 60 s synthesis deadline.

Scripted demonstration

```
python -m streamlit run researcher/web_ui.py
python -m researcher ask "photosynthesis" \
    --offline --sources wiki,arxiv --json
docker run --rm --network none vertex-research
```

Five original questions produce answers with numbered references.

Offline evidence is visibly labeled **SIMULATED**.

Live demo requires configured API keys. Docker build, offline run and UI health checks passed.

Testing evidence

114 passing tests

93.91% application
coverage

16 unchanged course smoke tests
included.

Ruff passes; mypy reports no issues in 17 files. Tests use mocks and deny live sockets.

High-value checks

- ▶ Source failure and timeout
- ▶ Semaphore bound and overlap
- ▶ 429 recovery; no 401 retry
- ▶ Cache expiry and corruption
- ▶ Invalid and missing citations

Tested extensions in v1.1.0

- ▶ **Web failover:** primary failure or timeout, then a configured secondary; chaos tests verify recovery.
- ▶ **Token budget:** estimated sliding-window reservations; exhaustion test observes 60 s backoff.
- ▶ **Streamlit:** same core pipeline, sample/live modes, references and downloads; AppTest and browser checks.
- ▶ **CI:** lint, types, tests, coverage and container UI health workflow.

Docker acceptance passed. Live Gemini and hosted CI remain separate checks. Bonus credit is assessed by the instructor.

Team and AI disclosure

Member	Defense preparation focus (proposed)
Yaqub Xəlilli	Orchestration, concurrency, benchmark
Səbuhi Xamiyev	AI boundary, retries, configuration
Aqil Əsgərov	Cache, tests, deployment

Codex substantially drafted the software, tests and documentation.

Actual ownership, PRs and commit shares must be recorded by the team. The contribution statement is unsigned.

Defense goal: every member can explain the full request path.

Questions

VerteX

`github.com/yagubbx/Research-Assistant`

Architecture, timeout behavior, citation validation and benchmark scope